

山东大学



作业五、作业六：全同态密码与应用实验

4×4 输入与 3×3 卷积核的密文卷积及旋转次数分析

成员姓名	学号	班级	主要分工
马悦曦	202300201057	网安 2 班	明文卷积校验、实验结果复核与报告整理
周思琪	202300450129	密码 1 班	密文卷积实现、CKKS 参数调研与编译运行验证
闫雨彤	202322161173	密码 1 班	打包布局设计、旋转索引检查与结果表格整理
孙世瑾	202300210010	密码 1 班	OpenFHE 环境配置、旋转次数优化与理论最小值分析

目录

1 实验概览	1
1.1 实验题目	1
1.2 实验目标	1
1.3 实验完成情况	1
1.4 实验环境	1
1.5 代码结构	2
2 实验原理	2
2.1 二维卷积计算	2
2.2 CKKS 打包布局	3
2.3 旋转与掩码	3
3 作业 5：密文卷积实现与正确性验证	3
3.1 OpenFHE 参数设置	3
3.2 直接密文卷积	4
3.3 正确性验证方法	4
4 作业 6：旋转次数优化与理论分析	5
4.1 直接法旋转次数	5
4.2 Baby-step/Giant-step 优化策略	5
4.3 理论最小值讨论	6
4.4 输出槽位压缩	7
5 实验结果	7
5.1 运行命令	7
5.2 运行输出	7
5.3 结果汇总	8
6 关键技术问题与解决方法	8
6.1 CKKS 近似误差的验证口径	8
6.2 旋转方向与槽位污染控制	8
6.3 BSGS 行掩码的槽位对齐	9
6.4 旋转次数统计口径	9
6.5 乘法深度与层级消耗	9
7 实验总结	9

1 实验概览

1.1 实验题目

本次实验包含两个连续作业。作业 5 要求使用 OpenFHE 库，在单输入单输出场景下，对一个 4×4 输入矩阵和一个 3×3 卷积核执行步长为 1、无填充的密文卷积，并验证密文计算结果与明文卷积结果在给定误差阈值内一致。作业 6 要求在上述卷积实现的基础上，采用“打包 $\rightarrow$ 旋转 $\rightarrow$ 累加”策略，统计本次实现中的密文旋转次数，并分析该次数在本实验计算模型下是否达到理论最小值。

1.2 实验目标

本实验的主要目标如下：

1. 理解 CKKS 近似同态加密方案中 SIMD 打包槽位、密文旋转和明文掩码乘法的作用；
2. 将 4×4 输入矩阵按行优先顺序打包到一个 CKKS 密文中，完成 3×3 卷积核的同态卷积；
3. 在密文端完成旋转、明文权重乘法和累加，解密后与明文卷积结果逐项比较；
4. 实现直接对角线法和 baby-step/giant-step 优化法两种密文卷积路径；
5. 统计两种方法的旋转索引和旋转次数，分析优化方法是否在本实验限定模型下达到理论最小旋转次数。

1.3 实验完成情况

实验使用 C++17 和 OpenFHE CKKS 接口完成。输入矩阵、卷积核和明文期望结果固定如下：

$$X = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix}, \quad K = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}.$$

卷积输出大小为 2×2 ，明文计算结果为：

$$Y = \begin{bmatrix} 348 & 393 \\ 528 & 573 \end{bmatrix}.$$

程序运行后，直接密文卷积、优化密文卷积和输出槽位压缩结果均通过误差验证。由于 CKKS 属于近似同态加密方案，若保留 8 位小数时误差显示为 0.00000000，只能表示误差小于当前打印精度，并不表示数学意义上的严格零误差。本实验以 10^{-4} 为误差阈值，三种结果的实际误差均低于该阈值，满足正确性要求。

1.4 实验环境

实验在 Ubuntu 虚拟机中编译运行，OpenFHE 由源码构建并安装到系统目录。为保证实验可复现，运行前记录了操作系统、内核、CPU、内存、编译工具链以及 OpenFHE 源码版本。核心环境如表 1 所示。

表 1: 实验软件环境

项目	配置
操作系统	Ubuntu 22.04 LTS
内核版本	Linux 5.15.0-139-generic, x86_64
CPU	AMD Ryzen 7 5800H with Radeon Graphics, 2 核, 1 线程/核
内存	10GiB RAM, 2.0GiB Swap
编译器	g++ 11.2.0
构建工具	CMake 3.22.1, GNU Make 4.3
同态加密库	OpenFHE
编程语言	C++17
实验代码目录	openfhe_conv_homework

1.5 代码结构

实验代码采用一个独立 CMake 工程组织，目录结构如下：

Listing 1: 实验代码目录结构

```
1 openfhe_conv_homework/  
2 |-- CMakeLists.txt  
3 `-- src/  
4     |-- main.cpp
```

其中，`src/main.cpp` 包含明文卷积、密文直接卷积、baby-step/giant-step 优化卷积、输出槽位压缩、解密验证和旋转次数统计等逻辑。`CMakeLists.txt` 负责查找并链接 OpenFHE 库。

2 实验原理

2.1 二维卷积计算

对于单输入单输出卷积，输入 X 大小为 4×4 ，卷积核 K 大小为 3×3 ，步长为 1 且无填充。因此输出矩阵大小为：

$$H_{\text{out}} = 4 - 3 + 1 = 2, \quad W_{\text{out}} = 4 - 3 + 1 = 2.$$

输出元素的明文卷积公式为：

$$Y_{i,j} = \sum_{u=0}^2 \sum_{v=0}^2 X_{i+u,j+v} K_{u,v}, \quad 0 \leq i, j < 2.$$

以 $Y_{0,0}$ 为例：

$$\begin{aligned} Y_{0,0} &= 1 \cdot 1 + 2 \cdot 2 + 3 \cdot 3 + 5 \cdot 4 + 6 \cdot 5 + 7 \cdot 6 \\ &\quad + 9 \cdot 7 + 10 \cdot 8 + 11 \cdot 9 \\ &= 348. \end{aligned}$$

同理可得 $Y_{0,1} = 393$, $Y_{1,0} = 528$, $Y_{1,1} = 573$ 。

2.2 CKKS 打包布局

CKKS 支持将多个实数或复数打包到同一个密文的不同槽位中，从而实现 SIMD 式并行计算。本实验将 4×4 输入矩阵按行优先顺序打包到 16 个槽位：

slot	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
value	$x_{0,0}$	$x_{0,1}$	$x_{0,2}$	$x_{0,3}$	$x_{1,0}$	$x_{1,1}$	$x_{1,2}$	$x_{1,3}$	$x_{2,0}$	$x_{2,1}$	$x_{2,2}$	$x_{2,3}$	$x_{3,0}$	$x_{3,1}$	$x_{3,2}$	$x_{3,3}$

卷积输出保留在原 4×4 布局的左上角槽位，即槽位 $[0, 1, 4, 5]$ 。这样可以避免为了显示 2×2 矩阵而引入额外的数据搬移；若需要将输出压缩到连续槽位 $[0, 1, 2, 3]$ ，程序额外执行一次布局旋转。

2.3 旋转与掩码

OpenFHE 中的 `EvalRotate(ct, r)` 可将打包密文槽位循环旋转。对于行优先布局，卷积核位置 (u, v) 对应的线性偏移为：

$$d = 4u + v.$$

将输入密文旋转 d 个槽位后，输出槽位上的数据就与卷积核位置 (u, v) 对应的输入元素对齐。随后使用明文掩码在输出槽位 $[0, 1, 4, 5]$ 处乘以卷积核权重 $K_{u,v}$ ，其他槽位置零，最后将 9 个加权项累加即可得到密文卷积结果。

3 作业 5：密文卷积实现与正确性验证

3.1 OpenFHE 参数设置

程序使用 CKKS 方案完成近似数值计算。主要参数为乘法深度 2、缩放模数位数 50、批处理槽位数 16。乘法深度设置为 2 可以覆盖密文与明文掩码乘法以及后续布局整理所需的层级。本实验卷积主体只使用密文与明文掩码相乘，因此 `EvalMultKeyGen` 不是卷积主体必需项；代码中保留该密钥生成语句主要是为了与 OpenFHE 常见 CKKS 示例写法保持一致，并不影响本实验结果。

Listing 2: CKKS 上下文与密钥生成核心代码

```

1  CCParams<CryptoContextCKKSRSNS> parameters;
2  parameters.SetMultiplicativeDepth(2);
3  parameters.SetScalingModSize(50);
4  parameters.SetBatchSize(16);
5
6  auto cc = GenCryptoContext(parameters);
7  cc->Enable(PKE);
8  cc->Enable(KEYSWITCH);
9  cc->Enable(LEVELEDSSHE);
10 cc->Enable(ADVANCEDSHE);
11
12 auto keys = cc->KeyGen();
13 cc->EvalMultKeyGen(keys.secretKey);
14 cc->EvalRotateKeyGen(keys.secretKey, {1, 2, 4, 5, 6, 8, 9, 10});

```

为了避免把零位移误计为一次旋转，程序封装了 `Rotate` 函数。当旋转索引为 0 时，函数直接返回原密文，不调用 `EvalRotate`，也不增加统计次数。

Listing 3: 零旋转不计数的封装函数

```

1 Ciphertext<DCRTPoly> Rotate(const CryptoContext<DCRTPoly>& cc,
2                               const Ciphertext<DCRTPoly>& ct,
3                               int32_t index,
4                               RotationStats& stats) {
5     if (index == 0) {
6         return ct;
7     }
8     ++stats.rotations;
9     stats.indices.push_back(index);
10    return cc->EvalRotate(ct, index);
11 }

```

3.2 直接密文卷积

直接法为每一个非零卷积核偏移构造一个旋转后的输入密文，再乘以对应权重掩码并累加。由于偏移 0 不需要旋转， 3×3 卷积核共有 9 个位置，因此直接法需要 8 次密文旋转。

$$C_{\text{direct}} = \sum_{u=0}^2 \sum_{v=0}^2 \text{Mask}_{[0,1,4,5]}(K_{u,v}) \cdot \text{Rot}(C_X, 4u + v).$$

其中 C_X 表示输入矩阵打包后的密文，`Rot` 表示密文旋转，`Mask` 表示只在输出槽位保留权重的明文掩码。

Listing 4: 直接密文卷积核心逻辑

```

1 for (size_t kh = 0; kh < 3; ++kh) {
2     for (size_t kw = 0; kw < 3; ++kw) {
3         int32_t offset = static_cast<int32_t>(4 * kh + kw);
4         auto shifted = Rotate(cc, ctInput, offset, stats);
5         auto mask = MakeMask(cc, {0, 1, 4, 5}, kernel[3 * kh + kw]);
6         auto term = cc->EvalMult(shifted, mask);
7         acc = first ? term : cc->EvalAdd(acc, term);
8     }
9 }

```

3.3 正确性验证方法

程序分别计算明文卷积、直接密文卷积和优化密文卷积，并比较对应输出槽位的数值。由于 CKKS 属于近似同态加密方案，编码、加密、密钥切换、旋转和明文乘法都可能引入微小数值误差，解密结果通常不应理解为数学意义上的精确整数。因此，本实验使用最大绝对误差作为正确性判断指标：

$$\epsilon_{\max} = \max_i |\hat{Y}_i - Y_i|.$$

程序中设置误差阈值为 10^{-4} 。若所有输出槽位误差均不超过阈值，则判定密文卷积结果通过正确性验证。运行结果中，如果最大误差在保留 8 位小数时显示为 0.00000000，只能说明误差小于当前

输出精度，不表示 CKKS 计算在数学意义上不存在误差。为了更清楚地观察微小误差，程序最终采用科学计数法和 15 位精度输出最大绝对误差。

4 作业 6：旋转次数优化与理论分析

4.1 直接法旋转次数

直接法使用卷积核的 9 个线性偏移：

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

偏移 0 对应原密文，不需要调用 `EvalRotate`，其余 8 个偏移均需要单独旋转，因此直接法的卷积旋转次数为：

$$R_{\text{direct}} = 9 - 1 = 8.$$

程序统计到的旋转索引为：

$$[1, 2, 4, 5, 6, 8, 9, 10].$$

4.2 Baby-step/Giant-step 优化策略

为了减少旋转次数，实验进一步实现了 baby-step/giant-step 结构。该策略先生成水平方向的三个对齐版本：

$$B_0 = C_X, \quad B_1 = \text{Rot}(C_X, 1), \quad B_2 = \text{Rot}(C_X, 2).$$

这一步只需要 2 次旋转。然后对每个卷积核行 u ，使用 B_0, B_1, B_2 和对应明文掩码构造该行的横向部分和 S_u 。三个行部分和分别位于相对输出槽位偏移 0, 4, 8 处，再将 S_1 旋转 4 个槽位、 S_2 旋转 8 个槽位并累加：

$$C_{\text{bsgs}} = S_0 + \text{Rot}(S_1, 4) + \text{Rot}(S_2, 8).$$

因此优化法总共需要：

$$R_{\text{bsgs}} = 2 + 2 = 4$$

次卷积旋转。

这里的掩码槽位必须随卷积核行号移动。设最终输出槽位集合为：

$$P = \{0, 1, 4, 5\}.$$

对于卷积核第 u 行，先使用的掩码槽位为：

$$P_u = \{p + 4u \mid p \in P\}.$$

因此三行对应：

$$P_0 = \{0, 1, 4, 5\}, \quad P_1 = \{4, 5, 8, 9\}, \quad P_2 = \{8, 9, 12, 13\}.$$

这样构造出的第 1 行和第 2 行部分和才能分别通过旋转 4 位和 8 位回到最终输出槽位。如果三行都直接在 $[0, 1, 4, 5]$ 上做掩码，然后再旋转 4 位、8 位，行部分和会被移动到错误槽位，结果将不正确。

Listing 5: BSGS 中每一行对应的掩码槽位

```
1 std::vector<size_t> RowShiftedOutputSlots(size_t kh) {  
2     const std::vector<size_t> outputSlots = {0, 1, 4, 5};  
3  
4     std::vector<size_t> result;  
5     for (size_t slot : outputSlots) {  
6         result.push_back(slot + 4 * kh);  
7     }  
8     return result;  
9 }
```

Listing 6: Baby-step/Giant-step 密文卷积核心逻辑

```
1 baby[0] = ctInput;  
2 baby[1] = Rotate(cc, ctInput, 1, stats);  
3 baby[2] = Rotate(cc, ctInput, 2, stats);  
4  
5 for (size_t kh = 0; kh < 3; ++kh) {  
6     auto rowSlots = RowShiftedOutputSlots(kh);  
7     for (size_t kw = 0; kw < 3; ++kw) {  
8         auto mask = MakeMask(cc, rowSlots, kernel[3 * kh + kw]);  
9         auto term = cc->EvalMult(baby[kw], mask);  
10        rowSum = firstCol ? term : cc->EvalAdd(rowSum, term);  
11    }  
12    auto alignedRow = Rotate(cc, rowSum, static_cast<int32_t>(4 * kh), stats);  
13    acc = firstRow ? alignedRow : cc->EvalAdd(acc, alignedRow);  
14 }
```

4.3 理论最小值讨论

以下理论下界只针对本实验采用的计算模型：输入矩阵按行优先顺序打包到单个 CKKS 密文中，不进行输入数据重复编码；卷积核的 9 个权重均非零；密文计算仅使用逐槽明文乘法、密文加法和循环旋转；输出保留在槽位 $[0, 1, 4, 5]$ ；统计的是在线 `EvalRotate` 调用次数；同时允许继续旋转已经累加过的中间密文。

在该模型中，一条从输入密文到输出密文的计算路径对应一个总旋转偏移。初始密文只包含偏移 0。每增加一次旋转，最多可以复制并平移当前已有的偏移集合，因此使用 R 次旋转能够形成的不同偏移数量至多为 2^R 。

完整的 3×3 稠密卷积需要以下 9 种输入偏移：

$$\{0, 1, 2, 4, 5, 6, 8, 9, 10\}.$$

因此旋转次数必须满足：

$$2^R \geq 9,$$

从而得到：

$$R \geq \lceil \log_2 9 \rceil = 4.$$

本实验的优化方法使用旋转索引 1, 2, 4, 8，共执行 4 次卷积旋转，因此在上述单密文、无重复编码的旋转-掩码-累加模型下达到了理论最小旋转次数。若允许改变编码方式，例如在更多槽位中

重复放置输入，或者使用多个输入密文，则可能用更少的在线旋转完成等价计算。因此，本文中的“理论最小值”不是所有密文卷积实现中的无条件最小值，而是本实验计算模型下的最小值。

4.4 输出槽位压缩

密文卷积自然输出在槽位 $[0, 1, 4, 5]$ ，与原始 4×4 矩阵的左上 2×2 位置一致。为了便于连续读取，程序还实现了将稀疏输出压缩到 $[0, 1, 2, 3]$ 的步骤。该步骤先保留槽位 $[0, 1]$ ，再保留槽位 $[4, 5]$ 并旋转 2 个槽位，最后相加：

$$C_{\text{compact}} = \text{Mask}_{[0,1]}(C) + \text{Rot}(\text{Mask}_{[4,5]}(C), 2).$$

该步骤需要 1 次额外旋转，但它属于结果布局整理，不计入卷积本身的旋转次数。

5 实验结果

5.1 运行命令

在 OpenFHE 安装完成后，实验工程使用如下命令编译并运行：

Listing 7: 编译与运行命令

```
1 cd ~/Desktop/openfhe_conv_homework
2 rm -rf build
3 mkdir build
4 cd build
5 cmake ..
6 make -j$(nproc)
7 ./openfhe_conv_homework
```

5.2 运行输出

程序输出的关键结果摘要如下。误差判断以 10^{-4} 为阈值；重新编译运行后，程序会以科学计数法输出实际最大绝对误差，具体数值可能随平台和 OpenFHE 参数有极小差异。

```
===== OpenFHE 密文卷积实验 =====
输入规模：4x4，卷积核：3x3，步长：1，无填充

明文卷积结果：
    348    393
    528    573

密文卷积结果（直接法）：
    348    393
    528    573

密文卷积结果（优化法）：
    348    393
    528    573

正确性验证：
直接法最大误差：2.387423592153937e-12，通过
优化法最大误差：2.728484105318785e-12，通过
连续槽位输出最大误差：2.216893335571513e-12，通过

旋转次数统计：
直接法：8 次，旋转索引 = [1, 2, 4, 5, 6, 8, 9, 10]
优化法：4 次，旋转索引 = [1, 2, 4, 8]
输出整理：1 次，旋转索引 = [2]

结论：
直接法需要 8 次卷积旋转。
在单密文、无重复编码、9 个权重均非零的模型下，理论下界为  $\text{ceil}(\log_2(9)) = 4$ 。
优化法需要 4 次卷积旋转，因此在上述模型下达到理论最小值。
输出从  $[0, 1, 4, 5]$  整理到  $[0, 1, 2, 3]$  还要额外 1 次旋转，但这一步不计入卷积计算。
```

图 1: 程序运行结果

5.3 结果汇总

表 2: 密文卷积正确性验证结果

方法	输出槽位	最大绝对误差	结果
直接对角线法	[0, 1, 4, 5]	$< 10^{-4}$	PASS
Baby-step/Giant-step 法	[0, 1, 4, 5]	$< 10^{-4}$	PASS
输出槽位压缩	[0, 1, 2, 3]	$< 10^{-4}$	PASS

表 3: 旋转次数统计

方法	旋转索引	旋转次数
直接对角线法	[1, 2, 4, 5, 6, 8, 9, 10]	8
Baby-step/Giant-step 法	[1, 2, 4, 8]	4
输出槽位压缩	[2]	1

6 关键技术问题与解决方法

6.1 CKKS 近似误差的验证口径

本实验的明文卷积结果都是整数，但密文端使用的是 CKKS 近似同态加密方案，编码、加密、旋转密钥切换、明文乘法和解密都会引入微小误差。因此，不能直接使用整数相等或字符串相等作为正确性判断标准。实验中采用最大绝对误差作为统一指标：

$$\epsilon_{\max} = \max_i |\hat{Y}_i - Y_i|.$$

程序设置阈值为 10^{-4} ，只要所有输出槽位误差低于该阈值，就认为密文卷积通过验证。为了避免把显示精度误认为数学意义上的精确零，程序最终使用科学计数法和 15 位有效数字输出误差：

Listing 8: CKKS 误差输出方式

```
1 std::cout << std::scientific << std::setprecision(15);
2 std::cout << "直接法最大误差: " << directErr << std::endl;
```

这样即使误差只有 10^{-12} 量级，也不会被简单打印成 0.00000000 而造成表述不严谨。

6.2 旋转方向与槽位污染控制

输入矩阵按行优先顺序打包，卷积核位置 (u, v) 对应线性偏移 $d = 4u + v$ 。OpenFHE 中正向 EvalRotate 会把后续槽位移动到当前槽位，因此旋转 d 位后，输出槽位 p 上对应的是原输入槽位 $p + d$ 。这正好与卷积窗口中 $X_{i+u, j+v}$ 的访问方式一致。

由于密文旋转是循环旋转，非输出位置可能出现跨行或回绕后的无关数据。实验通过明文掩码只保留 [0, 1, 4, 5] 等有效槽位，将其他槽位全部置零。这样既保证了输出位置对齐，也避免了无效槽位参与后续累加。

6.3 BSGS 行掩码的槽位对齐

优化法不是简单地对每个卷积核偏移分别旋转，而是先构造水平方向的三个 baby-step:

$$B_0 = C_X, \quad B_1 = \text{Rot}(C_X, 1), \quad B_2 = \text{Rot}(C_X, 2).$$

之后每一行的部分和需要先放在不同槽位集合中。设最终输出槽位为 $P = \{0, 1, 4, 5\}$ ，第 u 行对应的掩码槽位为：

$$P_u = \{p + 4u \mid p \in P\}.$$

因此三行分别为：

$$P_0 = \{0, 1, 4, 5\}, \quad P_1 = \{4, 5, 8, 9\}, \quad P_2 = \{8, 9, 12, 13\}.$$

完成各行横向加权求和后，再把第 1 行部分和旋转 4 位、第 2 行部分和旋转 8 位，使三行贡献重新对齐到最终输出槽位 $[0, 1, 4, 5]$ 。如果三行都直接使用 P 作为掩码，后续旋转会把行贡献移动到错误位置，这是 BSGS 实现中最容易出错的地方。

6.4 旋转次数统计口径

实验统计的是在线阶段实际调用 `EvalRotate` 的次数。偏移 0 不需要真正旋转，因此包装函数在 `index == 0` 时直接返回原密文，不增加计数：

Listing 9: 零旋转不计入统计

```
1 if (index == 0) {  
2     return ct;  
3 }  
4 ++stats.rotations;  
5 stats.indices.push_back(index);  
6 return cc->EvalRotate(ct, index);
```

按照这一口径，直接法的偏移集合为 $\{0, 1, 2, 4, 5, 6, 8, 9, 10\}$ ，其中 0 不计数，所以在线旋转次数为 8。优化法的有效旋转为 1, 2, 4, 8，在线旋转次数为 4。输出槽位压缩使用的旋转 2 属于结果布局整理，不计入卷积主体。

6.5 乘法深度与层级消耗

卷积主体只进行密文与明文掩码的逐槽乘法，没有密文与密文乘法。尽管如此，CKKS 中的明文乘法仍会消耗层级，并且输出槽位压缩阶段还会再次执行明文掩码乘法。因此程序将乘法深度设置为 2，使卷积主体和后续布局整理都有足够层级。各个卷积项都在相同的计算层级上生成，再进行密文加法累加，避免了不同层级密文直接相加带来的兼容性问题。

7 实验总结

本实验基于 OpenFHE 实现了单输入单输出的 4×4 输入与 3×3 卷积核密文卷积。程序将输入矩阵打包到一个 CKKS 密文中，通过旋转、明文掩码乘法和累加完成同态卷积。由于 CKKS 是近似同态加密方案，实验没有把解密结果描述为数学意义上的精确相等，而是采用最大绝对误差进行验证。直接法、优化法和连续槽位输出三条路径的误差均低于 10^{-4} 阈值，因此通过正确性验证。

在旋转次数方面，直接对角线法需要对除偏移 0 以外的 8 个卷积核位置分别旋转，因此需要 8 次卷积旋转。进一步采用 baby-step/giant-step 策略后，程序先复用水平方向的两个旋转，再对三个行部分和进行纵向对齐，只需要 4 次卷积旋转。该 4 次下界成立于本文限定的单密文、无重复编码、9 个权重均非零、只统计在线 `EvalRotate` 的旋转-掩码-累加模型下。

因此，本次实验不仅完成了作业 5 要求的密文卷积正确性验证，也完成了作业 6 要求的旋转次数探索。最终结论是：直接法 8 次旋转是受限对角线方法下的最小值；若允许旋转中间累加结果，则 baby-step/giant-step 方法以 4 次卷积旋转达到本实验计算模型下的理论最小值。若改变输入编码方式或使用多个输入密文，该结论不应直接推广为所有密文卷积实现的绝对下界。

参考文献

- [1] OpenFHE Development Team. *OpenFHE Documentation*. <https://openfhe-development.readthedocs.io/>.
- [2] OpenFHE Development Team. *OpenFHE: Open-Source Fully Homomorphic Encryption Library*. <https://github.com/openfheorg/openfhe-development>.
- [3] Jung Hee Cheon, Andrey Kim, Miran Kim, and Yongsoo Song. *Homomorphic Encryption for Arithmetic of Approximate Numbers*. ASIACRYPT 2017.
- [4] Shai Halevi and Victor Shoup. *Algorithms in HElib*. CRYPTO 2014.